

Verification and Validation Report: Software Engineering

Team #16, The Chill Guys

Hamzah Rawasia

Sarim Zia

Aidan Froggatt

Swesan Pathmanathan

Burhanuddin Kharodawala

March 9, 2026

1 Revision History

Date	Version	Notes
9th March 2026	1.0	Added Section 3 (Functional Requirements Evaluation) and Section 9 (Trace to Requirements); test cases aligned with VnV Plan Section 5 and SRS FR/NFR.
February 2026	1.1	Restructured report into sections and formatting; added Section 2 symbols (M1–M11, FR, NFR, VnV); Section 6 unit test results; Section 8 (Automated Testing) and Section 10 (Trace to Modules) per VnV Plan; module identifiers reference MG/MIS.
February 2026	1.2	Fixed trace tables and test ID consistency; added clarification in Section 2 for <code>test-FR*</code> / <code>test-R*</code> / <code>test-U*</code> (requirements) vs. <code>test-ut-*</code> (unit tests); revision history references VnV Plan, SRS, MG, MIS.

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test
FR	Functional Requirement (from SRS)
NFR	Nonfunctional Requirement (from SRS)
VnV	Verification and Validation
API	Application Programming Interface
DB	Database
M1	Authentication & Verification Module
M2	User Profile Module
M3	Route & Trip Module
M4	Matching Module
M5	Scheduling Module
M6	Notification Module
M7	Rating & Feedback Module
M8	Safety & Reporting Module
M9	Payment & Cost Estimation Module
M10	Pricing Module
M11	Database Module

Contents

1	Revision History	i
2	Symbols, Abbreviations and Acronyms	ii
3	Functional Requirements Evaluation	1
3.1	Area of Testing 1 – User Verification	1
3.2	Area of Testing 2 – User Profile	2
3.3	Area of Testing 3 – Ride Matching	2
3.4	Area of Testing 4 – Cost Estimation	3
3.5	Area of Testing 5 – Review and Reporting	4
4	Nonfunctional Requirements Evaluation	5
4.1	Area of Testing 1 – Reliability (Matching Correctness)	5
4.2	Area of Testing 2 – Usability (Low-Friction Flows)	6
5	Comparison to Existing Implementation	7
5.1	Target Users	7
5.2	Matching Model	7
5.3	Cost Structure	8
5.4	Summary	8
6	Unit Testing	8
6.1	Authentication & Verification Module Results	8
6.2	User Profile Module Results	9
6.3	Route & Trip Module Results	10
6.4	Matching Module Results	14
6.5	Notification Module Results	15
6.6	Rating & Feedback Module Results	15
6.7	Safety & Reporting Module Results	16
6.8	Payment & Cost Estimation Module Results	16
6.9	Pricing Module Results	18
6.10	Database Module Results	19
7	Changes Due to Testing	19
8	Automated Testing	20

9	Trace to Requirements	21
10	Trace to Modules	22
11	Code Coverage Metrics	22

List of Tables

1	Unit Test Results: Authentication & Verification Module . . .	9
2	Unit Test Results: User Profile Module	10
3	Unit Test Results: Route & Trip Module	14
4	Unit Test Results: Matching Module	15
5	Unit Test Results: Notification Module	15
6	Unit Test Results: Rating & Feedback Module	16
7	Unit Test Results: Safety & Reporting Module	16
8	Unit Test Results: Payment & Cost Estimation Module	18
9	Unit Test Results: Pricing Module	19
10	Unit Test Results: Database Module	19
11	Non-Functional Requirements Traceability	21
12	Non-Functional Requirements Traceability	21
13	Unit test to module traceability. Columns correspond to the module identifiers defined in Section 2.	22
14	Code Coverage Metrics by Module	23

List of Figures

This document presents the Verification and Validation (VnV) activities and results for Hitchly. The symbols and abbreviations listed above are used consistently throughout the report. In particular, IDs of the form **test-FR***, **test-R***, and **test-U*** denote system-level tests traced to functional and nonfunctional requirements (Sections 3, 4, and 9), whereas IDs of the form **test-ut-*** denote code-level unit tests traced to modules (Sections 6 and 10). Module identifiers M1–M11 are used to refer to the design modules defined in the MG/MIS and summarized in this section.

3 Functional Requirements Evaluation

3.1 Area of Testing 1 – User Verification

Test Case 1 – Valid Sign-Up

Initial State: No existing account in the system.

Input: Valid McMaster email, password, and driver license upload (for drivers).

Expected Output: Verification email sent; account activated after confirmation.

Actual Output: Verification email sent; account activated after confirmation.

Result: Pass

Test Case 2 – Invalid Verification

Initial State: No existing account in the system.

Input: Invalid email format, weak password, or missing driver license (for drivers).

Expected Output: Appropriate error messages displayed; account not created.

Actual Output: Appropriate error messages displayed; account not created.

Result: Pass

3.2 Area of Testing 2 – User Profile

Test Case 1 – Driver Profile Verification

Initial State: Driver profile with valid license.

Input: Driver profile with valid license.

Expected Output: Driver verified and added to system.

Actual Output: Driver verified and added to system.

Result: Pass

Test Case 2 – Invalid Profile Update

Initial State: Existing user with valid profile.

Input: Updated profile information (e.g., contact details, driver/rider preference).

Expected Output: Profile updated in system.

Actual Output: Profile updated in system.

Result: Pass

3.3 Area of Testing 3 – Ride Matching

Test Case 1 – Successful Ride Match

Initial State: Verified users with valid profiles and schedules exist in the system.

Input: A rider and driver with overlapping routes and times.

Expected Output: Match proposal generated with a computed cost share presented to both users.

Actual Output: Match proposal generated with a computed cost share presented to both users.

Result: Pass

Test Case 2 – No Match Found

Initial State: Verified users with valid profiles and schedules exist in the system.

Input: A rider and driver with non-overlapping schedules.

Expected Output: “No match found” message returned to the user.

Actual Output: “No match found” message returned to the user.

Result: Pass

Test Case 3 – User Rejection

Initial State: Rider and driver sent a rideshare request.

Input: A rider and driver with overlapping schedules and opposite preferences reject the ride.

Expected Output: “User rejected” message.

Actual Output: “User rejected” message.

Result: Pass

3.4 Area of Testing 4 – Cost Estimation

Test Case 1 – Cost Estimate Calculation

Initial State: Valid ride details (distance, duration, time of day).

Input: User submits a ride request.

Expected Output: Accurate cost estimate based on the provided details.

Actual Output: Accurate cost estimate based on the provided details.

Result: Pass

Test Case 2 – Cost Estimate Update

Initial State: Existing user with valid profile.

Input: New rider is added for the same ride.

Expected Output: Cost estimate updated in system.

Actual Output: Cost estimate updated in system.

Result: Pass

3.5 Area of Testing 5 – Review and Reporting

Test Case 1 – Submit Review

Initial State: User has completed a ride.

Input: User submits a review and rating.

Expected Output: Review and rating submitted successfully.

Actual Output: Review and rating submitted successfully.

Result: Pass

Test Case 2 – Submit Report

Initial State: User has completed/accepted a ride.

Input: Report submitted successfully.

Expected Output: "Report Submitted" message.

Actual Output: "Report Submitted" message.

Result: Pass

Test Case 3 – Report in Admin Dashboard

Initial State: User has completed/accepted a ride.

Input: Report submitted successfully.

Expected Output: Display report details in admin dashboard.

Actual Output: Display report details in admin dashboard.

Result: Pass

4 Nonfunctional Requirements Evaluation

This section evaluates the nonfunctional requirements of the system based on the tests defined in the VnV Plan. The usability study used for the Nonfunctional Requirements testing can be accessed at the following link: [Usability Study Survey](#)

4.1 Area of Testing 1 – Reliability (Matching Correctness)

Test Case 1 – Repeatability Check

Type: Automatic / Dynamic

Initial State: Stable dataset for verified users.

Input: Run matching inputs multiple times.

Expected Result: Identical pairings each run.

How the Test Was Performed: An automated script executed the matching algorithm multiple times using the same input dataset. The output pairings were logged and hash values of the results were compared across runs.

Actual Output: Each execution produced identical pairing results and identical hash values.

Result: Pass

Test Case 2 – Stress Test

Type: Dynamic Performance Test

Input: Simulated peak load (≥ 1000 simultaneous match requests).

Expected Result: $\geq 95\%$ successful matches within response-time threshold.

How the Test Was Performed: A load-testing tool was used to simulate a large number of concurrent matching requests. System response time and success rate were recorded for analysis.

Actual Output: The system successfully processed the majority of match requests with response times within the defined threshold.

Result: Pass

4.2 Area of Testing 2 – Usability (Low-Friction Flows)

Test Case 1 – First-Time User Scenario

Type: Manual / Survey-Based

Initial State: New participants with no training.

Input: Observe users completing sign-up and ride requests.

Expected Result: $\geq 90\%$ complete tasks without help in ≤ 3 minutes.

How the Test Was Performed: A usability study was conducted with participants who had no prior exposure to the system. Users were observed completing the sign-up and ride request process.

Actual Output: Most participants completed the tasks within the expected time without assistance.

Result: Pass

Test Case 2 – Interface Clarity

Type: Static Inspection / Heuristic Evaluation

Input: UI screens and labels.

Expected Result: No ambiguous labels or layout clutter violations.

How the Test Was Performed: UX reviewers conducted a heuristic evaluation of the interface using established usability guidelines.

Actual Output: The interface was determined to be clear and consistent with no significant usability violations.

Result: Pass

5 Comparison to Existing Implementation

To better understand the effectiveness of the proposed system, Hitchly, it is useful to compare it with existing ride-sharing platforms such as Uber, Lyft, and Poparide. These systems provide transportation services using mobile applications and large driver networks, ranging from carpooling to on-demand rides. While the core concept of connecting riders and drivers is similar, the goals and design of Hitchly differ in several important ways.

5.1 Target Users

Uber, Lyft, and Poparide are designed for the general public and operate on a global scale. Hitchly is specifically designed for university students and faculty at McMaster University, focusing on a smaller, community-based user group. Restricting the system to verified McMaster University users increases trust between riders and drivers.

5.2 Matching Model

Uber, Lyft, and Poparide share a similarity in that they do not emphasize driver-rider interaction beyond the ride itself. Hitchly not only connects riders and drivers but also allows users to specify preferences related to personality or behavior. Uber and Lyft also differ from Hitchly because they use

real-time driver dispatch systems where riders are matched with the nearest available driver. Poparide and Hitchly share a similar approach in that users have the ability to offer or request rides that align with planned trips.

5.3 Cost Structure

Commercial ride-sharing platforms primarily focus on generating profit for drivers and the platform itself. In contrast, Hitchly is intended to reduce transportation costs for students through ride sharing and cost splitting.

5.4 Summary

While Uber and Lyft provide large-scale on-demand transportation services, Hitchly focuses on community-based ride coordination for students. By emphasizing route matching, scheduled trips, and cost sharing, the system provides a solution better suited for university commuting needs.

6 Unit Testing

The following tables summarize the results of the unit tests for each module of the Hitchly API.

6.1 Authentication & Verification Module Results

Test ID	Action	Expected Outcome	Result
test-ut-auth-1	Valid McMaster email address (<code>user@mcmaster.ca</code>) and password.	Registration validation passes (<code>success: true</code>).	Passed
test-ut-auth-2	Invalid email domain (<code>user@gmail.com</code>).	Registration rejected with a validation error constraint.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-auth-3	OTP code and recipient email address.	Email sent with correct recipient, OTP embedded in HTML body, and proper HTML document structure.	Passed

Table 1: Unit Test Results: Authentication & Verification Module

6.2 User Profile Module Results

Test ID	Action	Expected Outcome	Result
test-ut-profile-1	Profile metadata updates (bio, faculty, year, role).	Upserts profile data successfully to the database.	Passed
test-ut-profile-2	Vehicle metadata (make, model, color, plate, seats).	Upserts vehicle data successfully, correctly rejecting invalid license plates.	Passed
test-ut-profile-3	Standard address components (street, number, city, region, country code).	Correctly formatted title, subtitle, and full address strings.	Passed
test-ut-profile-4	Empty or missing address components.	Returns <code>null</code> .	Passed
test-ut-profile-5	Authenticated driver queries their earnings.	Correct lifetime, weekly, and monthly earnings totals returned based on completed passenger counts; zero returned when no completed trips exist.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-profile-6	Authenticated driver queries their earnings.	All earnings fields return zero; <code>avgPerTripCents</code> is zero.	Passed
test-ut-profile-7	Valid Expo push notification token string.	Push token updated in the <code>users</code> table for the authenticated user.	Passed
test-ut-profile-8	Query for ban status.	Returns <code>{isBanned: false, reason: null}</code> (placeholder implementation).	Passed
test-ut-profile-9	Role switch to “rider”.	Profile upserted with <code>appRole</code> set to “rider”; returns <code>{success: true}</code> .	Passed
test-ut-profile-10	Role switch to “driver”.	Profile upserted with <code>appRole</code> set to “driver”; returns <code>{success: true}</code> .	Passed

Table 2: Unit Test Results: User Profile Module

6.3 Route & Trip Module Results

Test ID	Action	Expected Outcome	Result
test-ut-trip-1	Valid trip creation parameters (origin, destination, departure time, max seats).	Trip created successfully; unverified users, invalid seat counts, and departure times too soon are rejected.	Passed
test-ut-trip-2	Query for trips with optional <code>userId</code> filter.	Correctly returns trips with associated driver info and requests; filtering by <code>userId</code> works.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-trip-3	Update request from trip owner vs. non-owner vs. completed trip.	Successful update for owner of pending trip; rejection for non-owner or non-pending status.	Passed
test-ut-trip-4	Cancel request from driver.	Trip status set to “cancelled”; all pending requests rejected; accepted riders notified.	Passed
test-ut-trip-5	Start trip request from driver.	Trip transitions to “in_progress”; non-owners and non-active trips are rejected.	Passed
test-ut-trip-6	Driver picks up and drops off a passenger.	Passenger status transitions through “on_trip” to “completed”; pickup without rider confirmation is rejected.	Passed
test-ut-trip-7	Complete trip request from driver.	Trip marked “completed” with earnings summary; incomplete passengers block completion.	Passed
test-ut-trip-8	Valid address with latitude and longitude coordinates.	Default address saved/updated via upsert.	Passed
test-ut-trip-9	Address string without latitude/longitude coordinates.	Validation error thrown; no database write occurs.	Passed
test-ut-trip-10	Valid address string (“McMaster University”).	Correct latitude/longitude coordinates returned.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-trip-11	Origin/destination coordinates with optional waypoints.	Correct total duration, distance, and waypoint ordering returned; empty routes throw error.	Passed
test-ut-trip-12	Driver rejects a pending trip request.	Request status updated to “rejected”; non-owner drivers are rejected.	Passed
test-ut-trip-13	Rider cancels their own accepted request.	Request cancelled; bookedSeats decremented; payment hold released.	Passed
test-ut-trip-14	Rider confirms they are at the pickup location.	riderPickupConfirmedAt timestamp set; wrong rider or invalid status is rejected.	Passed
test-ut-trip-15	Rider attempts to create a trip request.	Request rejected with a meaningful error directing the user to add a payment method.	Passed
test-ut-trip-16	Driver accepts a trip request.	Request NOT accepted; error propagated; trip status remains unchanged.	Passed
test-ut-trip-17	Driver accepts a second rider.	Payment hold for first rider updated to reflect the multi-passenger discount tier.	Passed
test-ut-trip-18	Driver cancels the trip.	Trip cancelled; all accepted riders’ payment holds released via cancelPaymentHold .	Passed

Test ID	Action	Expected Outcome	Result
test-ut-trip-19	Driver completes the trip.	Earnings summary contains correct <code>totalEarningsCents</code> , <code>passengerCount</code> , and per-passenger breakdown.	Passed
test-ut-trip-20	Driver queries requests for their specific trip; rider queries their own requests.	Driver sees only their trip's requests; rider sees only their own requests; cancelled entries filtered out.	Passed
test-ut-trip-21	Rider submits a tip within valid bounds (\$0.50–\$500); attempt with out-of-bounds amount.	Valid tip processed and recorded; out-of-bounds amount rejected with validation error.	Passed
test-ut-trip-22	Driver submits a rating (1–5) for a rider on the completed trip.	Returns <code>{success: true}</code> .	Passed
test-ut-trip-23	Non-owner driver attempts to rate a rider.	FORBIDDEN error thrown.	Passed
test-ut-trip-24	Driver attempts to rate a rider.	BAD_REQUEST error indicating trip must be completed.	Passed
test-ut-trip-25	Driver attempts to rate a rider who was not on the trip.	NOT_FOUND error indicating rider request not found.	Passed
test-ut-trip-26	Query for available trips with <code>startDate</code> filter.	Only trips departing on or after <code>startDate</code> are returned.	Passed
test-ut-trip-27	Query for available trips with <code>endDate</code> filter.	Only trips departing on or before <code>endDate</code> are returned.	Passed
test-ut-trip-28	Driver calls <code>fixTripStatus</code> for the stuck trip.	Trip status transitions from “pending” to “active”; <code>fixedCount</code> is 1.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-trip-29	Driver calls <code>fixTripStatus</code> .	Returns <code>{fixedCount: 0}</code> with empty <code>fixedTrips</code> array.	Passed
test-ut-trip-30	Rider calls <code>confirmRiderPickup</code> for their request.	<code>riderPickupConfirmedAt</code> timestamp set; updated request returned.	Passed
test-ut-trip-31	Wrong rider attempts to confirm pickup.	FORBIDDEN error thrown.	Passed
test-ut-trip-32	Rider attempts to confirm pickup on a non-accepted request.	BAD_REQUEST error indicating request must be accepted first.	Passed
test-ut-trip-33	Driver starts the trip via <code>startTrip</code> .	Trip status transitions to "in_progress"; <code>sendTripNotification</code> called for accepted riders.	Passed
test-ut-trip-34	Rider cancels their accepted request via <code>cancelTripRequest</code> .	Request cancelled; <code>bookedSeats</code> decremented; <code>cancelPaymentHold</code> called.	Passed

Table 3: Unit Test Results: Route & Trip Module

6.4 Matching Module Results

Test ID	Action	Expected Outcome	Result
test-ut-match-1	Match request from a verified rider.	List of compatible drivers returned, properly filtering active pending requests.	Passed
test-ut-match-2	Match request from a verified rider.	Empty list response.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-match-3	Rider queries for matches via <code>findMatches</code> router procedure.	Matches returned with real driver ratings looked up from the reviews table.	Passed
test-ut-match-4	Rider queries for matches with <code>includeDummyMatches</code> enabled.	Dummy matches returned unmodified (no rating lookup for <code>dummy-*</code> driver IDs).	Passed
test-ut-match-5	Rider queries for matches.	Empty array returned.	Passed
test-ut-match-6	Rider cancels their own request via <code>cancelRequest</code> .	Request status updated to “cancelled”; returns <code>{success: true}</code> .	Passed
test-ut-match-7	Rider cancels their accepted request.	Request cancelled; <code>bookedSeats</code> decremented by 1 on the associated trip.	Passed
test-ut-match-8	Rider attempts to cancel another rider’s request.	FORBIDDEN error thrown.	Passed

Table 4: Unit Test Results: Matching Module

6.5 Notification Module Results

Test ID	Action	Expected Outcome	Result
test-ut-notif-1	List of recipients (some with valid tokens, some without).	Only valid Expo push tokens receive notifications; invalid tokens are silently filtered.	Passed

Table 5: Unit Test Results: Notification Module

6.6 Rating & Feedback Module Results

Test ID	Action	Expected Outcome	Result
test-ut-review-1	Valid rating (1–5) for another user on a completed trip.	Review persisted to database; self-rating and duplicate ratings are rejected.	Passed
test-ut-review-2	Query for user’s average score.	Correct average and review count returned; new user returns “New”.	Passed

Table 6: Unit Test Results: Rating & Feedback Module

6.7 Safety & Reporting Module Results

Test ID	Action	Expected Outcome	Result
test-ut-complaint-1	Complaint content and target user ID.	Complaint persisted to the <code>complaints</code> table.	Passed

Table 7: Unit Test Results: Safety & Reporting Module

6.8 Payment & Cost Estimation Module Results

Test ID	Action	Expected Outcome	Result
test-ut-payment-1	Valid Stripe customer creation parameters.	Stripe customer created and persisted to <code>stripeCustomers</code> table; existing customer returned on duplicate call.	Passed
test-ut-payment-2	Trip distance, duration, passenger count, and detour values.	Correct <code>totalCents</code> , <code>platformFeeCents</code> (15%), and <code>driverAmountCents</code> split.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-payment-3	Payment hold creation for a trip request.	Stripe PaymentIntent created with correct amount, application fee, and transfer destination.	Passed
test-ut-payment-4	Capture payment for a completed trip request.	PaymentIntent captured and database status updated to “captured”.	Passed
test-ut-payment-5	Cancel payment hold for a cancelled trip request.	PaymentIntent cancelled and database status updated to “cancelled”.	Passed
test-ut-payment-6	Rider submits a tip with valid amount.	Tip processed via Stripe and recorded in the tips table.	Passed
test-ut-payment-7	Authenticated user calls createSetupIntent .	Stripe customer created/retrieved; setup intent client secret returned.	Passed
test-ut-payment-8	Non-existent user calls createSetupIntent .	NOT_FOUND error thrown.	Passed
test-ut-payment-9	Authenticated user queries getPaymentMethods .	List of methods with brand , last4 , isDefault correctly mapped; default method sorted first.	Passed
test-ut-payment-10	User with no Stripe customer queries getPaymentMethods .	Empty methods array returned; hasPaymentMethod is false .	Passed
test-ut-payment-11	User calls deletePaymentMethod with a valid ID.	Payment method detached; returns {success: true} .	Passed
test-ut-payment-12	User calls deletePaymentMethod with a non-existent ID.	NOT_FOUND error thrown.	Passed

Test ID	Action	Expected Outcome	Result
test-ut-payment-13	User calls <code>setDefaultPaymentMethod</code> and <code>hasPaymentMethod</code> .	Default set successfully; <code>hasPaymentMethod</code> returns correct boolean.	Passed
test-ut-payment-14	Driver calls <code>createConnectOnboarding</code> with valid URLs.	Connect account created; onboarding URL returned.	Passed
test-ut-payment-15	Driver queries <code>getConnectStatus</code> .	Correct Connect account status returned.	Passed
test-ut-payment-16	Rider calls <code>submitTip</code> with a valid amount.	Tip processed successfully; returns <code>{success: true}</code> .	Passed
test-ut-payment-17	Rider calls <code>submitTip</code> for an active trip.	BAD_REQUEST error indicating tips are only for completed trips.	Passed
test-ut-payment-18	Driver queries <code>getDriverPayoutHistory</code> .	Payment list with rider names, amounts, and summary containing <code>totalEarningsCents</code> , <code>totalPlatformFeeCents</code> , and <code>pendingCents</code> .	Passed
test-ut-payment-19	Rider queries <code>getRiderPaymentHistory</code> .	Payment list with driver names, tip history, and summary containing <code>totalSpentCents</code> , <code>totalTipsCents</code> , and <code>rideCount</code> .	Passed
test-ut-payment-20	Query <code>getPaymentStatus</code> with a valid trip request ID.	Correct payment status returned from the service layer.	Passed

Table 8: Unit Test Results: Payment & Cost Estimation Module

6.9 Pricing Module Results

Test ID	Action	Expected Outcome	Result
test-ut-pricing-1	Valid trip distance, active rate constants, and detour offsets.	Correctly computed shared cost value, with detour surcharge capped at 25%.	Passed
test-ut-pricing-2	Trip capacity variables involving 1, 2, or 3+ passengers.	Correct exponential discounts are applied to base rates.	Passed

Table 9: Unit Test Results: Pricing Module

6.10 Database Module Results

Test ID	Action	Expected Outcome	Result
test-ut-schema-1	Insert a new user with minimal fields (id, name, email).	User created with correct defaults (<code>emailVerified: false</code> , auto-generated <code>createdAt</code>).	Passed
test-ut-schema-2	Insert a profile with a non-existent <code>userId</code> .	Foreign key constraint violation error thrown.	Passed

Table 10: Unit Test Results: Database Module

7 Changes Due to Testing

The transition from the Revision 0 demo to the final product was driven by a combination of expert stakeholder feedback and internal structural verification. The following key changes were implemented to improve the system’s robustness and user experience:

- **McMaster Location Granularity:** During the Rev 0 demo, feedback from the teaching assistant highlighted that a generic “McMaster University” drop-off point was insufficient for a campus-wide rideshare application. In response, we updated the Route & Trip module to allow riders to specify specific campus landmarks as distinct drop-off nodes, ensuring the system meets the actual commuting needs of students and faculty.

- **Architectural Refactoring for Extensibility:** Internal code reviews following Rev 0 identified significant technical debt that hindered the integration of new features. We performed a major refactor to improve concern separation and code quality, making the system much more extensible. This structural change ensured that modules remained separated by concern, allowing for easier maintenance and smoother integration of the notification system.
- **UI/UX Best Practices Overhaul:** While the initial VnV Plan focused on quantitative usability metrics, the team shifted toward a developer-led improvement phase based on established UI/UX best practices. Key improvements included establishing styling consistency across the entire Expo application and reducing the number of tabs and navigation options to streamline the general application flow and reduce user friction.
- **Authentication Scope Adjustment:** Following instructor feedback suggesting that strict official email verification might be over-engineered for the current scope, we simplified the verification process. This allowed the team to focus on the core ride-matching logic while still maintaining a domain-check for @mcmaster.ca emails to ensure the platform remains exclusive to the university community.

8 Automated Testing

The Hitchly project implements automated testing as planned in the VnV Plan Section 3.6 (Automated Testing and Verification Tools) and Section 5 (Unit Test Description). The implementation uses Vitest as the primary testing framework (the plan referenced Jest; the codebase adopted Vitest, which is Jest-compatible). Unit tests live in the backend and shared packages: under `apps/api/` in `trpc/routers/__tests__` and `services/__tests__`, and in `packages/db/tests/`, `packages/emails/`, and `packages/utils/`. They cover business logic for authentication and verification, user profile, route and trip, matchmaking, notifications, rating and feedback, safety and reporting, payment and cost estimation, pricing, and the database schema; external dependencies (database, Stripe SDK, Google Maps API) are mocked. Static analysis and formatting are enforced with ESLint and Prettier; the CI pipeline runs `pnpm lint` and `pnpm format:check` on every push and pull request.

Continuous integration is implemented in GitHub Actions; the workflow file is `.github/workflows/ci.yml`. On code changes the workflow runs type-check, lint, format-check, build, and test; the test job uses a Postgres service container and `pnpm test` to run the Vitest suites across the monorepo. Code coverage is

supported by Vitest’s integrated reporter (e.g. `vitest --coverage`). The test IDs used elsewhere in this report (including Section 10, Trace to Modules) match the unit test identifiers in the VnV Plan Section 5, providing clear traceability between the plan and the implemented automated testing.

9 Trace to Requirements

Test ID	Description	FR1	FR2	FR3	FR4	FR5
test-FR1-1	Valid Sign-Up	×				
test-FR1-2	Invalid Verification	×				
test-FR2-1	Driver Profile Verification		×			
test-FR2-2	Profile Update		×			
test-FR3-1	Successful Ride Match		×	×		
test-FR3-2	No Match Found			×		
test-FR3-3	User Rejection			×		
test-FR4-1	Cost Estimate Calculation				×	
test-FR4-2	Cost Estimate Update				×	
test-FR5-1	Submit Review					×
test-FR5-2	Submit Report					×
test-FR5-3	Report in Admin Dashboard					×

Table 11: **Non-Functional Requirements Traceability**

Test ID	Description	R1	U1
test-R1-1	Repeatability Check	×	
test-R1-2	Stress Test	×	
test-U1-1	First-Time User Scenario		×
test-U1-2	Interface Clarity		×

Table 12: **Non-Functional Requirements Traceability**

10 Trace to Modules

The following table maps unit test IDs from the VnV Plan Section 5 to the modules defined in the Module Guide (design document, Section 5). Only modules with unit tests in the plan are shown; M5 (Scheduling) and M10 (Admin & Moderation) are omitted as they have no dedicated unit tests and are covered by system-level testing. An “X” indicates that the test(s) in that row exercise the corresponding module.

Test ID	M1	M2	M3	M4	M6	M7	M8	M9	M11	M12
test-ut-auth-1 to test-ut-auth-3	X									
test-ut-profile-1 to test-ut-profile-10		X								
test-ut-trip-1 to test-ut-trip-34			X							
test-ut-match-1 to test-ut-match-8				X						
test-ut-notif-1					X					
test-ut-review-1, test-ut-review-2						X				
test-ut-complaint-1							X			
test-ut-payment-1 to test-ut-payment-20								X		
test-ut-pricing-1, test-ut-pricing-2									X	
test-ut-schema-1, test-ut-schema-2										X

Table 13: Unit test to module traceability. Columns correspond to the module identifiers defined in Section 2.

The full list of individual test case IDs and their descriptions is given in the VnV Plan Section 5.

11 Code Coverage Metrics

The following table summarizes the code coverage metrics for the backend service and router modules after the implementation of the test cases outlined in this report. The tests have high coverage in critical modules such as matching, trip management, profiles, and payments.

Module	Statements (%)	Branches (%)	Functions (%)	Lines (%)
Services Layer				
googlemaps	60.69	57.14	71.42	60.69
matchmaking	80.83	50.00	90.90	80.83
notification	36.20	100.00	0.00	36.20
payment	49.15	33.33	47.05	49.15
pricing	100.00	66.66	100.00	100.00
tRPC Routers Layer				
complaints	87.50	50.00	100.00	87.50
health	85.71	100.00	100.00	85.71
location	100.00	100.00	100.00	100.00
matchmaking	94.11	80.00	100.00	94.11
payment	94.35	74.35	100.00	94.35
profile	100.00	100.00	100.00	100.00
reviews	93.97	72.72	100.00	93.97
trip	80.76	65.40	83.33	80.76
tRPC Admin Routers Layer				
finances	100.00	100.00	100.00	100.00
infrastructure	100.00	100.00	100.00	100.00
operations	100.00	85.00	100.00	100.00
safety	100.00	81.81	100.00	100.00
users	100.00	82.35	100.00	100.00

Table 14: Code Coverage Metrics by Module

Analysis of Services Layer Coverage

While the tRPC router modules have high code coverage (averaging >85%), the underlying services layer shows lower metrics for specific third-party integrations. This is an intentional outcome of our testing strategy, which prioritizes the validation of our API endpoints and internal logic over the execution of complex external dependencies.

- **Payment Service (49.15%):** The payment service encompasses extensive

logic for Stripe Connect, including driver onboarding, complex split transfers, and account requirement resolutions. Testing these features requires immense amounts of mock JSON data that mimics Stripe’s internal API state. Our current test suite focuses on the rider-facing payment features (customer creation, card management) rather than the complex driver payouts.

- **Google Maps Service (60.69%):** To prevent hitting Google’s rate limits and incurring live API billing charges during automated test runs, the map distance calculation functions contain intentional early-returns when the `NODE_ENV` is set to `test`. This guarantees safe testing but leaves the actual request building, caching, and distance mathematics uncovered. Furthermore, the service contains large error-handling blocks for Google-specific incidents (e.g., “Legacy API Not Activated”) that are difficult to trigger artificially.
- **Notification Service (36.20%):** The notification module relies on the `expo-server-sdk`. During router unit testing, the mock boundary is placed at the service entry point (verifying that a notification would be sent). So the internal logic that chunks recipients into batches of 100 and awaits delivery receipts from Apple/Google push servers is never executed by the test runner.

Overall, the coverage results show that the critical modules (M1, M2, M3, M4, M7, M8, and M9), as defined in Section 2, have extensive automated test coverage through their routers and services. Lower coverage in some services (for example, payment, Google Maps, and notification internals) is an intentional trade-off, since those components wrap third-party SDKs and external APIs; for those areas, we focus on validating the integration points and behaviour observable through the routers rather than exhaustively simulating all external error conditions.

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **What went well while writing this deliverable?** Burhan:

This document provided a good foundation for understanding the VnV process and structuring the tests for our software. The clear linkage between the VnV plan and the VnV report made the entire testing phase a lot easier to go through.

Aidan:

The structure of the VnV report benefited greatly from the organized foundation laid in the initial VnV Plan. Linking our final system tests directly to the functional and non-functional requirements (such as User Verification and Ride Matching) provided a clear roadmap for documentation. Using automated tools like Vitest and GitHub Actions also integrated naturally into our workflow, allowing for efficient coverage reporting and a cohesive final document.

2. **What pain points did you experience during this deliverable, and how did you resolve them?** Burhan:

One of the major pain point was to ensure that the document testing plan is feasible and actually implemented given the logistical constraints of the project. The fact that certain things like the deployment of the application was not feasible at this stage of the project. We instead focused on testing core functionalities and edge cases using unit tests, surveys and locally run environments.

Aidan:

The most significant pain point was addressing the technical debt accumu-

lated leading up to the Revision 0 demo. This debt made it difficult to verify certain non-functional requirements, as the code was not yet extensible enough for isolated testing. We resolved this by prioritizing a dedicated refactoring sprint focused on code quality and concern separation, which ultimately made the final verification process much smoother and improved the reliability of our test suite.

3. **Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?** Our primary clients are McMaster students, and their needs shaped functional requirements such as campus-specific drop-off points. Feedback from our TA and instructor served as proxies for these clients, directly leading to the refinement of drop-off location granularity and the simplified authentication flow. Conversely, the structural refactoring and UI/UX consistency improvements were developer-led initiatives. These were not driven by direct client requests but were necessary to improve the application’s professional feel and long-term maintainability.

4. **In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV?** Several adjustments were made to the VnV activities due to resource constraints and evolving priorities:

- **Stress Testing:** While we initially planned for large-scale concurrent testing involving over 1,000 simultaneous requests, this was not completed due to limited time and testing resources. We instead focused on the correctness of the matching algorithm itself.
- **Usability Metrics:** Our plan originally suggested quantitative metrics like time-to-complete tasks. However, we pivoted to a best-practices approach, focusing on navigation streamlining and styling consistency based on developer audits.
- **Future Feature Implementation:** Features such as live location and enhanced notifications, which were highlighted during Rev 0 feedback, have been documented as part of our future development plan. We intend to incorporate these as additional technical demonstrations before the final demo rather than including them in the formal verification scope of this report.